Especificação do Projeto 2 - SD

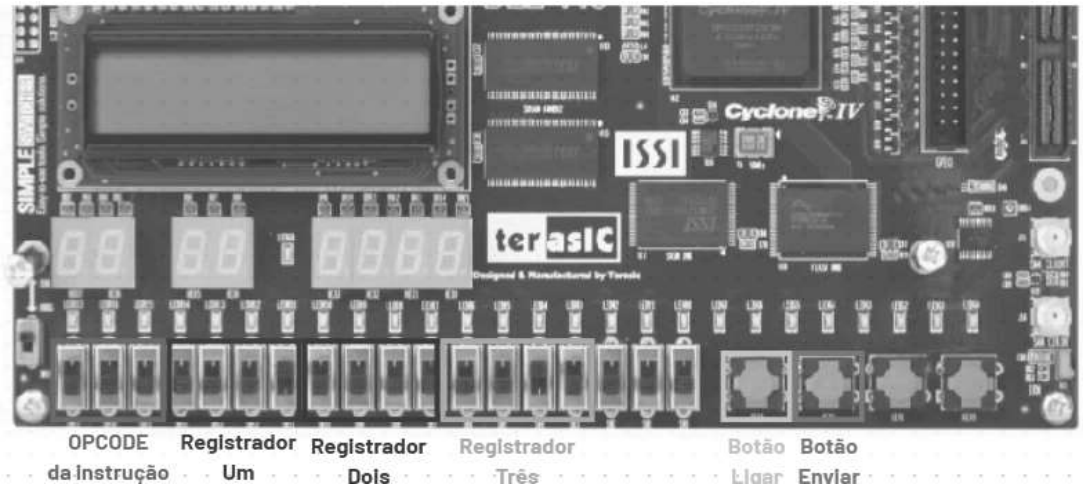
Descrição

Implementar uma Mini_CPU que contém um banco de memória virtual criado em verilog, uma unidade lógica aritmética (ULA) que irá lidar com os diferentes tipos de instruções recebidas pelas CPU e utilizando o LCD da placa DE2-115 para mostrar quais os resultados das instruções.

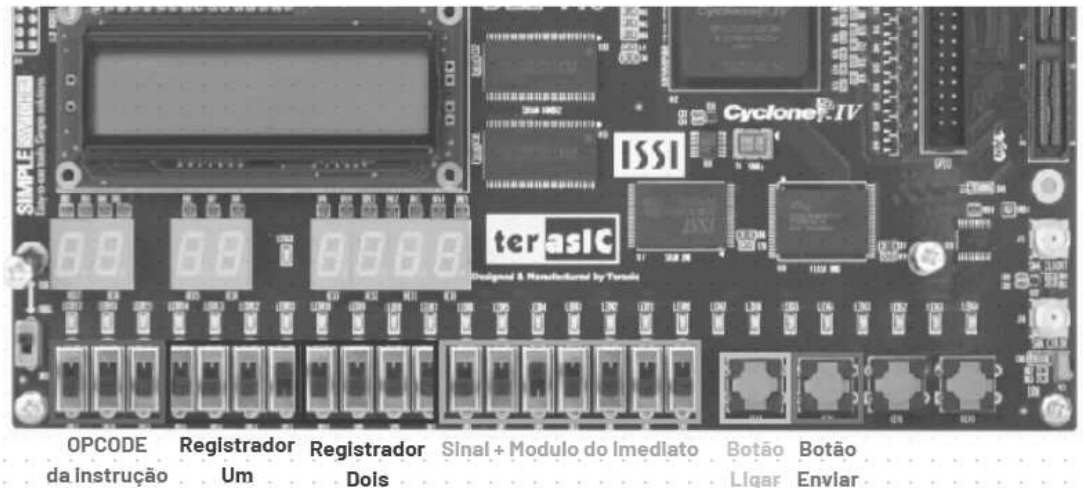
Inicialmente, o projeto usará dois botões (Que só devem realizar suas respectivas ações uma única vez assim que forem **soltos**) e todos os *switches* da placa (Ver a seção Inputs e Tipos de Instruções ou o esquema abaixo para mais informações). As instruções só podem ser enviadas quando o sistema for ligado, elas serão selecionadas usando os switches da placa e enviadas para processamento ao **soltar** o botão de "envio".

Ao final do uso do sistema, realizando ou não múltiplas instruções, os registradores devem ser todos zerados com a reinicialização.

Mapeamento de Inputs - Instruções Reg x Reg

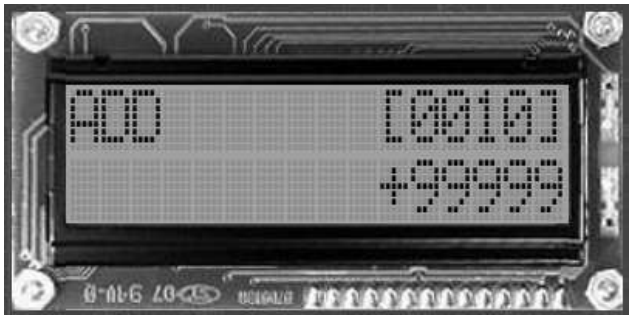


Mapeamento de Inputs - Instruções Reg x Imediatos



O LCD

No LCD, serão mostradas não só quais registradores foram usados para a realização da operação, como também **qual** operação foi realizada, qual o registrador de destino e, nele, o resultado final da operação incluindo o sinal de cada número. Como demonstra a figura ao lado.



Como vocês já praticaram em laboratório, o *display* irá usar um *data bus* de 8 bits e 3 sinais de controle para leitura e escrita. Vocês podem ver mais sobre eles no *datasheet* da placa. Vale lembrar, também, que o *display* é uma peça complementar a placa, e portanto, possui o seu próprio tempo para realizar funções. Portanto, após cada instrução ser enviada com o *negedge* (1 → 0) do sinal de controle *ENABLE* é recomendado esperar cerca de **1ms** para o sinal se propagar.

Vocês podem encontrar mais informações no datasheet do *LCD* com os devidos comandos de instruções e caracteres, é recomendado também testar cada instrução.

Função	**RS**	**RW**	**Data**							
			D7	D6	D5	D4	D3	D2	D1	D0
Limpar display	0	0	0	0	0	0	0	0	0	1
Modo de entrada	0	0	0	0	0	0	0	1	I/D	S
Ligar/Desligar Display	0	0	0	0	0	0	1	D	C	B
Modo funcional	0	0	0	0	1	DL	N	F	-	-
Ir para Posição	0	0	1	Endereço						
Escrever caractere	1	0	Código de Caractere							

- I/D:(1) - O cursor move para a direita ao escrever o caractere; (0) - move para a esquerda.
- S:(1) - Shift do Display ativado; (0) - Desativado.
- D:(1) - Display ligado; (0) - Display desligado.

- C: (1) - Cursor piscando ativado; (1) - Cursor piscando ativado.
- B: (1) - Cursor piscando ativado; (0) - Cursor piscando desativado.
- DL: (1) - Transmissão de Data de 8 bits; (0) - Transmissão de 4 bits.
- N: (1) - 2 Linhas; (0) 1 Linha.
- F: (1) - Caracteres 5x10; (0) - Caracteres 5x8.

Comportamento Esperado

Usem essa sessão para se basear na forma como o projeto deve funcionar, não tenham medo de colocar coisa a mais do que está aqui, só tenham em mente que tudo que estará descrito aqui é o mínimo esperado.

Inicialização



Ao passar o programa de vocês para a placa, o sistema deve estar "desligado", ou seja, nada deve ser mostrado no *display*, nenhuma instrução pode ser enviada, nenhum processamento, que consiste em leitura da instrução, interpretação da instrução, acesso à memória, escrita e leitura dos registradores, ocorre durante esse estado. Assim que o botão de ligar for apertado e solto o *display* é inicializado conforme demonstrado na imagem ao lado. Nesse estado, todos os registradores no módulo de memória se encontram zerados e prontos para uso.

Enviando Instruções

Com o display ligado e o sistema pronto, as instruções já podem ser enviadas para processamento. As informações das instruções serão indicadas através dos *switches* conforme a figura de **mapeamento de inputs** na descrição do projeto e nas Especificações de input. Elas só devem ser enviadas para processamento após o botão Enviar ser apertado e solto, nesse ponto, ela deve ser propriamente processada e seu resultado deve aparecer no *display* conforme está descrito na seção de Especificações de LCD.

Desligar

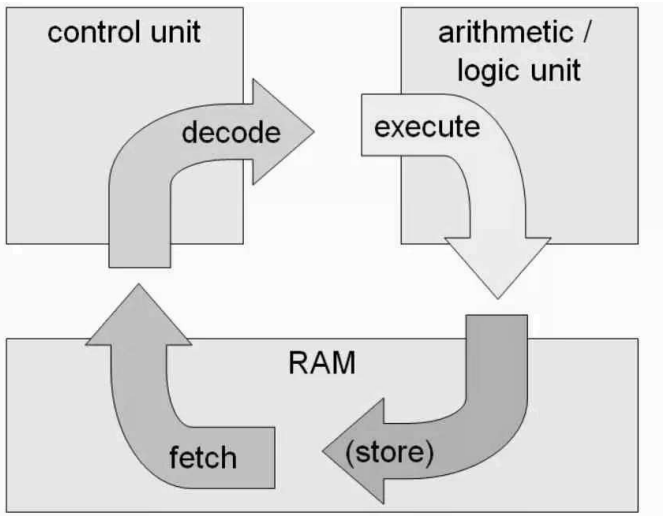
Se o botão de ligar for pressionado e solto novamente o display deve ser apagado e todas as informações gravadas nos registradores devem ser zeradas para a próxima vez que o sistema for iniciado.

Especificações

Use essa seção para se inteirar dos requisitos para cada parte do projeto. Novamente, cada parte dessa seção é obrigatória, mas coisas novas podem ser adicionadas ao projeto.

Inputs e Tipos de Instruções

Os inputs para o circuito de vocês será dado através dos *swicthes* da placa FPGA DE-115, e uma entrada de 18 bits será enviada após o pressionar do "botão de enviar", onde, logo após isso, o sistema deve ser responsável por realizar a busca, decodificação e execução da instrução indicada.



O sistema de vocês está responsável por lidar com 8 instruções divididas em 4 tipos, cada uma das instruções terá o seu próprio *opcode* único que equivale à parte da instrução que identifica qual operação está sendo realizada. São elas:

Instrução	*opcode*	Instrução	*opcode*
LOAD	000	SUBI	100
ADD	001	MUL	101
ADDI	010	CLEAR	110
SUB	011	DISPLAY	111

Tipo 1 – Operações Aritméticas com Registradores

Essas instruções operam exclusivamente com registradores.

- **ADD D1, O1, O2**
  - **Descrição**: Soma os valores armazenados nos registradores O1 e O2 e armazena o resultado no registrador de destino D1.
- **SUB D1, O1, O2**
  - **Descrição**: Copia o valor armazenado no registrador O1 para o registrador de destino D1.

Os inputs desse tipo de instrução só estão interessado nos 15 bits mais significativos da sua entrada e se organizam desse modo:

[14:12] Opcode	[11:8] Dest	[7:4] Src1	[3:0] Src2
Example: ADD R6, R7, R8 → 001_0110_0111_1000			

Tipo 2 – Operações Aritméticas com Imediatos

Essas instruções realizam operações utilizando um valor imediato (`Imm`).

- **ADDI D1, O1, `Imm`**
  - **Descrição**: Soma o valor armazenado no registrador O1 com o valor imediato `Imm` e armazena o resultado no registrador de destino D1.
- **SUBI D1, O1, `Imm`**

- **Descrição**: Subtrai o valor imediato `Imm` do valor armazenado no registrador `O1` e armazena o resultado no registrador de destino `D1`.

- **MUL D1, O1, Imm**
  - **Descrição**: Multiplica o valor armazenado no registrador `O1` pelo valor imediato `Imm` e armazena o resultado no registrador de destino `D1`.

[17:15] Opcode	[14:11] Dest	[10:7] Src1	[6] Sinal	[5:0] Immediato
Example: ADDI R6, R5, 25				

Tipo 3 – Operações de Carga

Essas instruções manipulam dados entre os registradores e valores imediatos.

- **LOAD D1, Imm**
  - **Descrição**: Copia o valor imediato `Imm` e armazena no registrador de destino `D1` presente na memória de dados.

[13:11] Opcode	[10:7] Dest	[6] Sinal	[5:0] Immediato
Example: LOAD R6, 25			

Tipo Especial – Controle e Saída

Essas instruções realizam operações específicas para manipulação de estado e exibição de dados.

- **CLEAR**
  - **Descrição**: Zera todos os 16 registradores no banco de registradores. Deve aparecer somente o nome da operação no display.
- **DISPLAY O1**
  - **Descrição**: Exibe o valor armazenado no registrador `(R1, R2... R15)`.

[7:4] Opcode	[3:0] Src1
Example: CLEAR→ 010_——- Example: DISPLAY 02 → 111_0010	

LCD

Cada tipo de Instrução deve ser mostrada de uma certa maneira no display a fim de mostrar de forma clara qual foi o resultado da operação realizada.

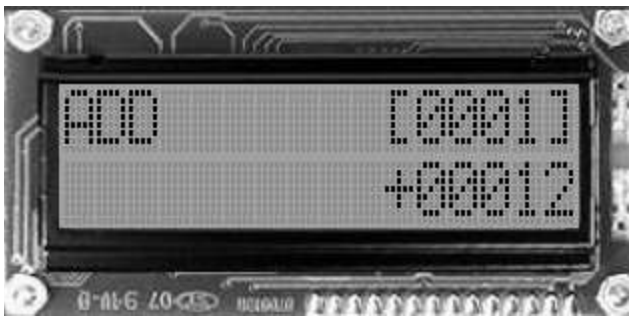
Modelo para Instruções do tipo Registrador

Operação Realizada	Número do Registrador de Destino em Binário
	Valor no Reg

Exemplo:

- LOAD R2 12
- ADD R1 R0 R2

Resultado Esperado:



Modelo para Instruções do tipo Imediato

Operação	Número do Registrador de Destino em Binário
	Valor no Reg

Exemplo:

- ADDI R1 R0 12

Resultado Esperado:



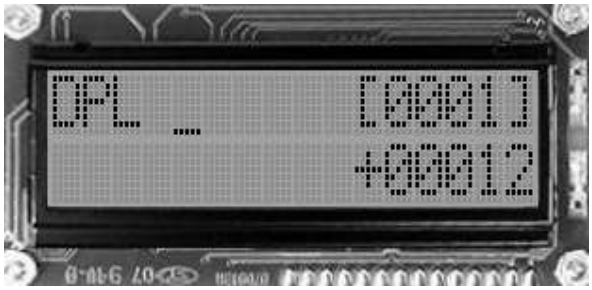
Modelo para Instruções Especiais

Operação	Número do Registrador de Destino em Binário (Apenas a instrução Display)
	Valor no Reg (Apenas na Instrução Display)

Exemplo:

- LOAD R2 12
- DISPLAY R2

Resultado Esperado:



Módulos

O projeto deve ter 3 módulos, separados e obrigatórios, novos módulos podem ser adicionados caso necessário. São eles:

1. module_mini_cpu.v
2. module_alu.v
3. memory.v

Fortemente recomendado que vocês criem uma máquina de estado para cada um, isso vai ajudar vocês a visualizar o fluxo de dados dentre os módulos.

Registradores e Memória

O programa deve incluir uma memória de dados com um design de **RAM de 16 × 16 bits**, ou seja, **16 registradores de 16 bits cada um**. O sistema precisa gerenciar eficientemente o fluxo de informações, diferenciando operações de leitura e escrita. Além disso, deve ser capaz de armazenar os dados conforme são recebidos, garantindo que as instruções sejam processadas corretamente.

Exemplo:

Ao iniciar o sistema, nossa memória de dados se encontra totalmente zerada e pronta para uso, vamos utilizar um código básico para ilustrar o funcionamento:

- LOAD R1, 10 // Armazena o valor 10 no registrador R2

Registrador	Valor (16 bits)
R0	0000 0000 0000 0000
R1	0000 0000 0000 1010
R2	0000 0000 0000 0000
R3	0000 0000 0000 0000
...	...
R15	0000 0000 0000 0000

- ADDI R2, R1, 5 // Soma 5 ao valor no registrador R1 e escreve em R2

Registrador	Valor (16 bits)
R0	0000 0000 0000 0000
R1	0000 0000 0000 1010
R2	0000 0000 0000 1111
R3	0000 0000 0000 0000
...	...
R15	0000 0000 0000 0000